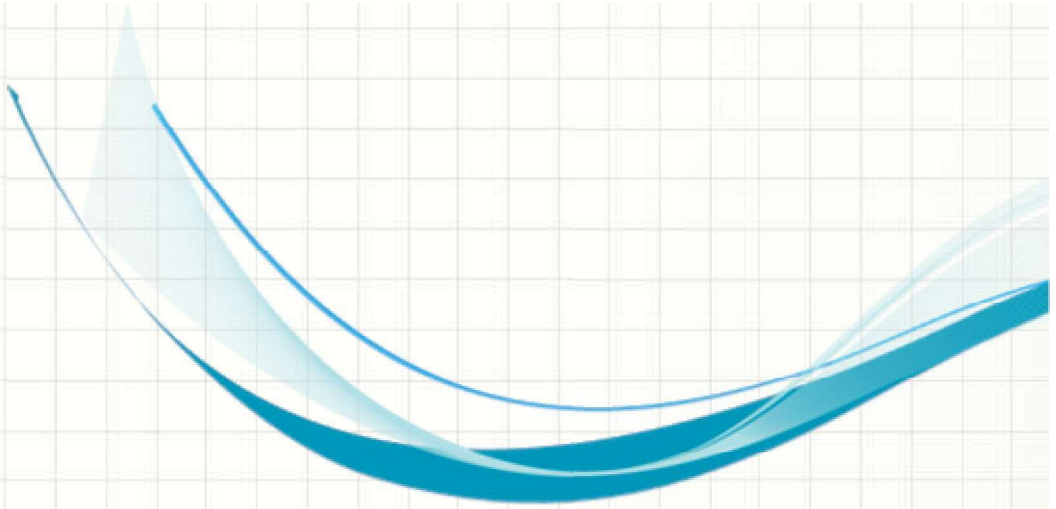


Teoría de Objetos

Teoría de Objetos

- **Qué es un Paradigma**
  - Es una visión de la computación, una forma arbitraria de definir qué es el software.
  - Concebir el soft de determinada forma, te permite tener una comprensión de lo que estás haciendo al desarrollar, ir armando un criterio de cosas que conviene hacer y otras que conviene no hacer

Objetos, ¿Para qué y por qué?

- **No por sus cualidades estéticas, moda, etc.**
- **Objetivo: soft que cumpla ciertas cualidades**

  Nuestro objetivo como profesionales es construir software que cumpla con determinadas cualidades

  - Escabilidad en carga y en funcionalidad
  - Robustez
  - Mantenibilidad
  - etc.

Los Tres Conceptos Básicos

- **Objeto**
  - ¿Qué es un objeto, qué merece ser llamado “objeto”?

  Objeto: Ente que tiene comportamiento

  Ente: Es un ente computacional, que está dentro del software, puede representar o no un ente de la realidad que está afuera de la computadora

  Tiene comportamiento nada más?

Los Tres Conceptos Básicos

- **Objeto**
  - Redefinimos entonces

Objeto: Ente computacional que exhibe comportamiento

Comportamiento: son las cosas que sabe el objeto

Exhibe: quiere decir que se las puedo pedir (quién?), desde afuera del objeto se puede acceder a ese comportamiento.

Los Tres Conceptos Básicos

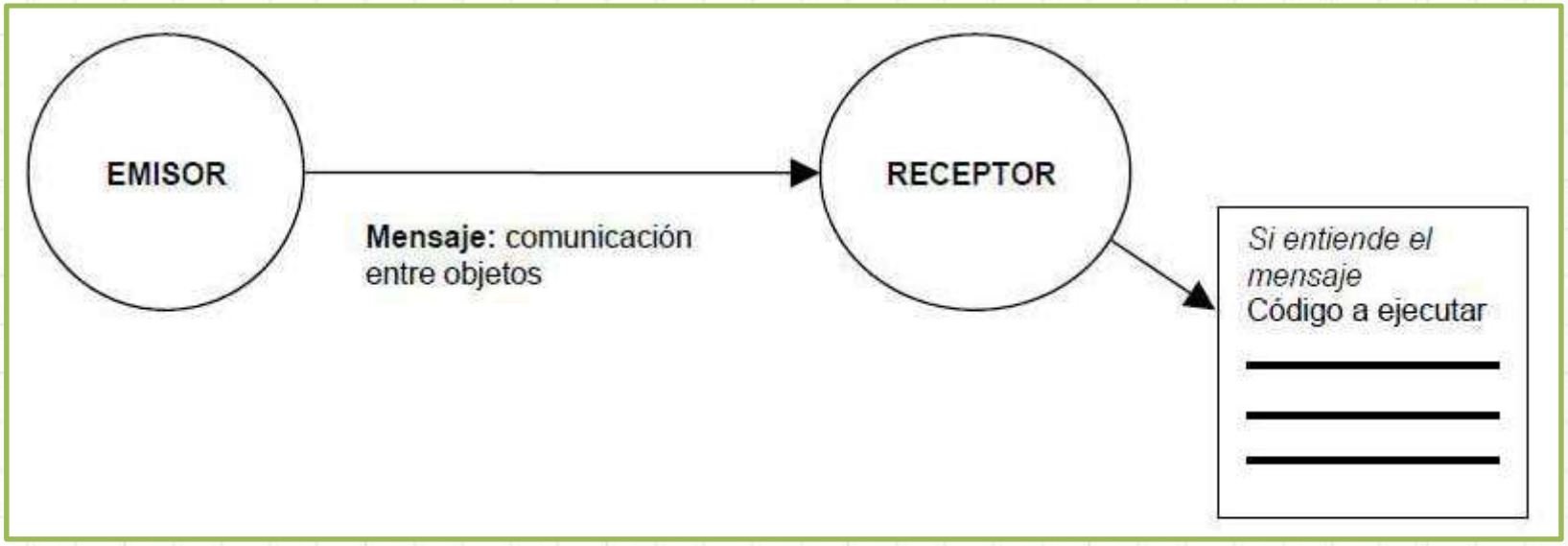
- **Mensaje**
  - En una aplicación OO, entonces, tenemos objetos que se piden cosas entre sí utilizando “Mensajes”

  Comunicación entre dos objetos emisor E y receptor R.

  E le pide algo a R y puede o no obtener algo como respuesta.

Los Tres Conceptos Básicos

- **Mensaje**
  - Un objeto ¿entiende cualquier mensaje que se le envía?
  - Cuando un objeto recibe un mensaje que sí entiende, ¿qué pasa?



Los Tres Conceptos Básicos

- **Ambiente**
  - ¿Dónde viven los objetos?
  - Para dar cuenta del habitat de los objetos definimos

Ambiente: lugar donde viven los objetos.

  - Crear un objeto = agregarlo al ambiente, destruirlo = sacarlo del ambiente.

Juntando todo

- **Software**
  - Con estos tres conceptos podemos decir qué es el software

  Software: resultado de la interacción de un conjunto de objetos que viven en un ambiente y que se comunican entre sí enviándose mensajes.

Juntando todo

- **Software**
  - **Entonces al diseñar Software debemos definir:**
    - qué objetos vamos a incluir en la aplicación
    - qué mensajes va a entender cada objeto
    - cuáles son los patrones de interacción entre los distintos objetos
    - cómo se van a conocer los objetos entre ellos

- **Preguntas…**
  - ¿Qué mensajes entiende un objeto? y ¿qué pasa cuando le intento enviar a un objeto un mensaje que no va a entender?
  - Cuando un objeto recibe un mensaje, se ejecuta código. ¿Qué código, dónde lo escribo?
  - ¿Cómo se crea un objeto?

Clase y Método

- **Método**
  - **Un método es la sección de código que se ejecuta al enviarse un mensaje.**
  - **Se identifica con una firma**

  - ¿Puede haber varios métodos con la misma firma?
  - **"Binding dinámico":** la selección se hace en tiempo de ejecución, en el momento que el objeto recibe el mensaje. **Depende del receptor, el emisor no tiene idea de qué método se va a ejecutar.**

Clase y Método

- **Clase**
  - El molde a partir del cual se crean los objetos
    - Generalización de comportamiento y datos comunes a objetos
    - Define qué características tiene que tener un nuevo objeto
    - Se dice que un objeto es instancia de una clase si fue creado usando esa clase como molde.
      - ¿qué son las instancias? Los objetos
      - ¿de cuántas clases es instancia un objeto? De exactamente una.
  - El ente que determina el comportamiento de sus instancias
    - Contiene los métodos que se van a ejecutar cuando una instancia reciba un mensaje.
    - Define los mensajes que entenderá el objeto

Clase y Método

- **Clase**
  - Los mensajes que va a entender un objeto son los que tienen métodos en su clase (“su clase” =“la clase de la cual es instancia el objeto”) (Polimorfismo)
  - ¿Todos los mensajes que puede entender un objeto tienen un método en su clase? No, herencia

Clase y Método

- **Clase Abstracta**
  - Una clase abstracta es una clase que se supone que no debería tener instancias
  - Método abstracto: se declara pero no se define en una clase, para que en los descendientes se defina, en cada subclase como corresponda.
  - Cualquier clase con algún método abstracto debe ser abstracta

La utilidad está en el código que usa las características abstractas, no en las características en sí

Clase y Método

- **Herencia**
  - Permite compartir, extender y redefinir comportamiento y datos entre clases.
  - Si dijimos que una clase es a la vez una generalización y un molde para objetos, una subclase será un molde un poco más específico.
  - La herencia organiza y facilita el polimorfismo y el encapsulamiento
  - Si una clase “Hereda” de más de una clase hablamos de herencia múltiple (no admitida en C#)
  - Al Subclasear podemos redefinir los metodo heredados (según visibilidad)

Polimorfismo

- **Lo usamos para lograr distinto comportamiento (según el objeto receptor) enviando el mismo mensaje**
  - Al enviar un mensaje puede que no sepa, y en principio no me debería importar, qué método se va a ejecutar.
  - Es posible que en distintas corridas del mismo código, el mismo mensaje resulte en la ejecución de métodos distintos
  - Me permite potencialmente agregar nuevas clases que respeten el contrato y los clientes (métodos que usan el mensaje) pueden no enterarse de que quién recibe el mensaje es distinto a lo que antes recibía
  - Ejemplos

Interfaces

- Una interface es un conjunto de declaraciones de métodos, sin código
- Parecida a una clase abstracta en la que todos los métodos son abstractos
- Son tipos, o sea que se pueden declarar variables de tipo interface, y parámetros de tipo interface, y el tipo de retorno de un método puede ser una interface. Pero no están en la jerarquía de herencia de clases.
- Tienen una jerarquía de herencia entre ellas, se puede decir que una interface hereda de otra

Interfaces y Herencia Múltiple

- Las interfaces de Java brindan una de las características de la herencia múltiple.
- Una clase no puede heredar de muchas clases, pero puede implementar muchas interfaces
- Entonces sus instancias (o las de sus descendientes) van a tener muchos tipos, y van a poder ser usados en cualquier lugar donde se declare alguno de esos tipos.
- Una instancia de una Clase A la podré ver como todas las instancias que dicha clase implementa

Qué se gana con las interfaces

- Despegamos el polimorfismo de la jerarquía de herencia de clases.
- Recordemos que la característica más valiosa del polimorfismo es que en un contexto no sé qué método se va a ejecutar al enviarse un mensaje.
- Sin interfaces, eso es cierto pero tengo una limitación: el receptor tiene que descender de la clase que declaro en el código.

Las interfaces permiten explotar más el concepto de polimorfismo

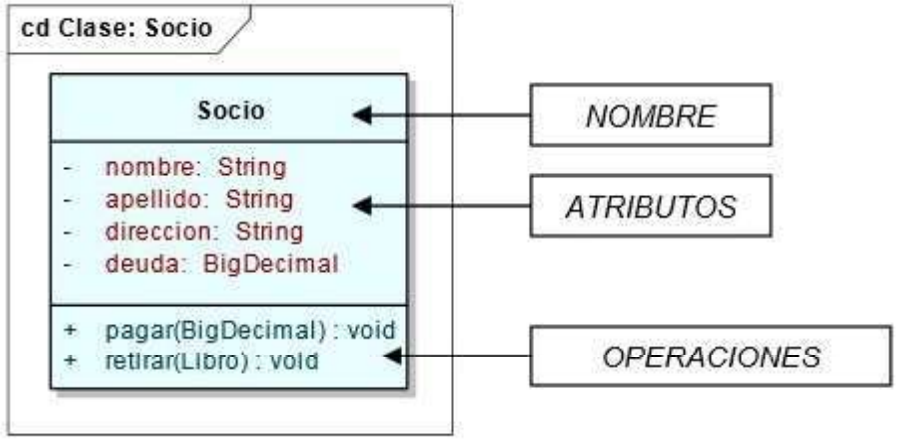
Bases de la Programación Orientada a Objetos

- **Abstracción:** proceso mental de extracción de las características esenciales de algo, ignorando los detalles superfluos.
- **Encapsulación:** proceso por el que se ocultan los detalles del soporte de las características esenciales de una abstracción.
- **Modularización:** proceso de descomposición de un sistema en un conjunto de módulos o piezas independientes y cohesivos (con significado propio). Lo adecuado es conseguir los mínimos acoplamientos.
- **Jerarquización:** proceso de estructuración por el que se produce una organización (jerarquía) de un conjunto de elementos en grados o niveles de responsabilidad, incumbencia o composición entre otros.

Diagrama de Clases

Diagrama de Clases

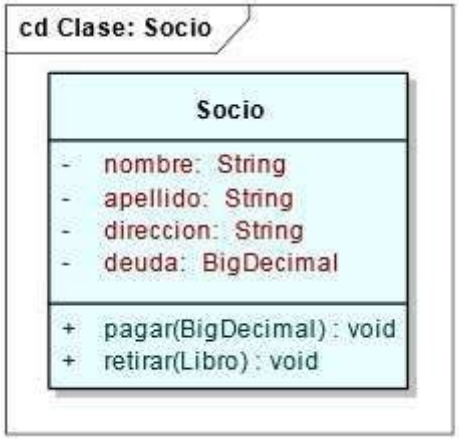
- El diagrama de clases es una herramienta que permite modelar las relaciones entre las entidades.
- En UML, una clase es representada por un rectángulo que posee tres divisiones:



Visibilidad
+ public
- private
protected

Clases

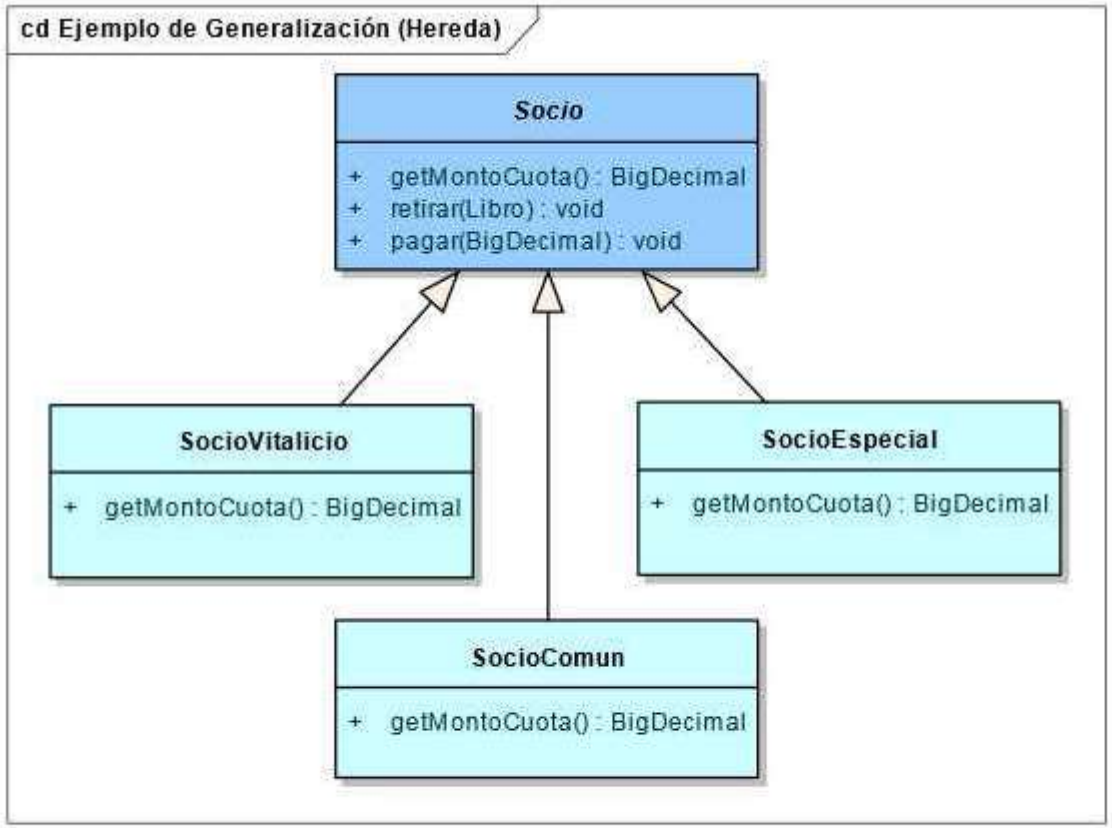
Antes que nada: un socio no es un socio...



Es frecuente pensar: *"A mí me parece mal que el Socio tenga un atributo deuda, porque en la realidad no es el socio el que sabe su deuda, es la Biblioteca la que registra cuánta plata debe el socio..."*

El modelo no es la realidad y nuestro objetivo no es reflejar la realidad tal cual, sino tener una simplificación de esa realidad infinitamente compleja que resuelva un problema particular (requerimiento).

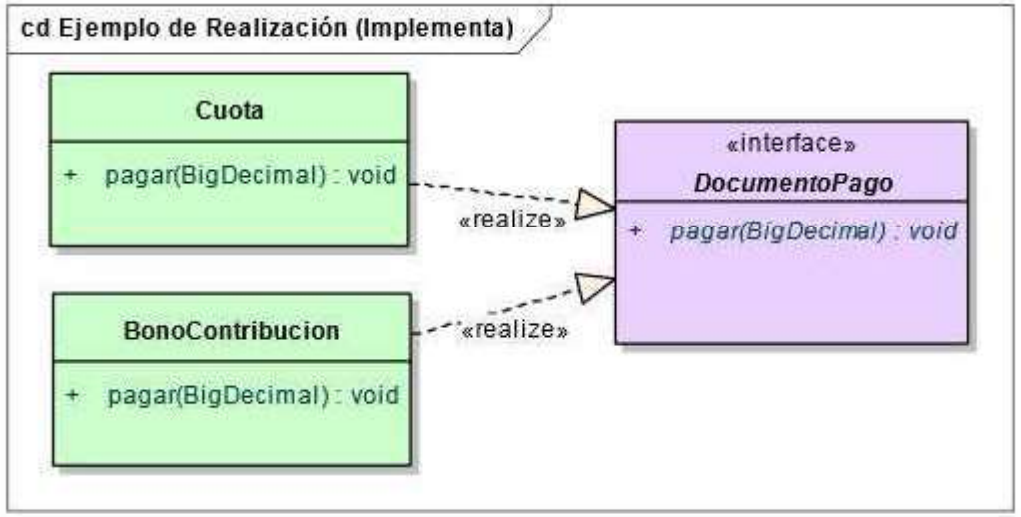
RELACION “HEREDA”



Generalización: una clase específica hereda los *atributos, relaciones, operaciones y métodos* de otra más general.

Cuando una subclase redefine el comportamiento de su superclase, se escriben los nombres de los métodos que redefine (el SocioVitalicio, por ejemplo, redefine el método getMontoCuota()).

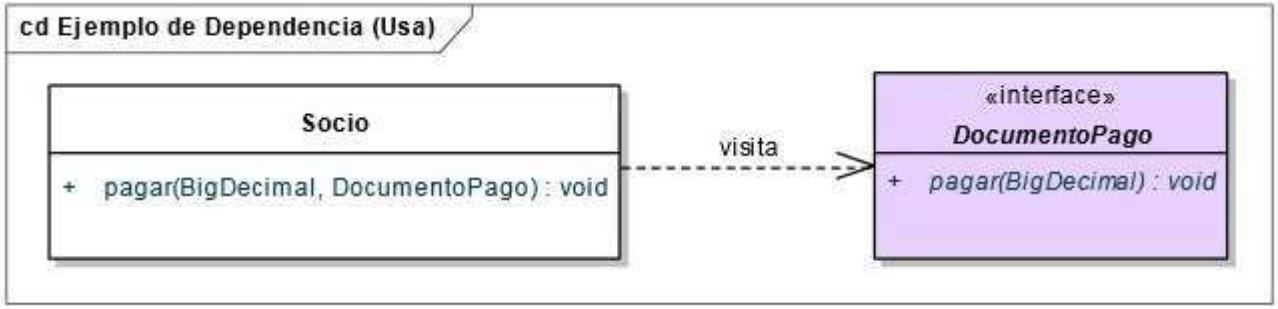
RELACIÓN “IMPLEMENTA”



Realización: se establece entre una clase y una interfaz; esto implica que la clase debe implementar todas las operaciones que defina la interfaz.

En el ejemplo Cuota y BonoContribucion son documentos de pago que deben definir una operación *pagar* (cada uno lo implementa de diferente manera).

RELACIÓN “USA”



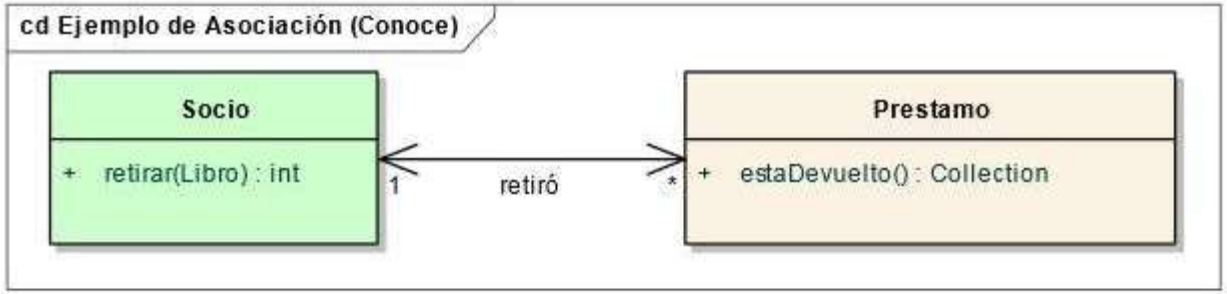
Dependencia: uno de los elementos usa o depende del otro cuando:

- El socio recibe o devuelve como parámetro de alguno de sus métodos un documento de pago
- Si el socio instancia un objeto que implementa documento de pago (pero no lo almacena como variable de instancia, sólo vive como variable local en el contexto de un método)

Indica que los dos elementos colaboran entre sí, pero que esa relación es débil, casual

RELACIÓN “CONOCE” (asociación)

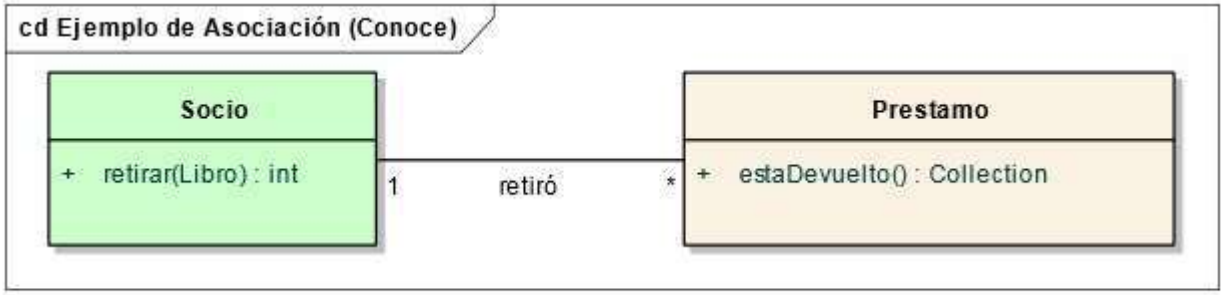
- Asociación: uno de los elementos conoce al otro, almacenándolo como variable de instancia.
- Puede definirse una etiqueta que expresa el rol que cumple dicha relación. En cada extremo de la asociación puede agregarse la siguiente información:



- Puede ponerse un nombre de rol a la asociación (retiró)
- Pueden ponerse las flechas de navegabilidad, en el ejemplo anterior ambas clases pueden llegar al otro miembro de la asociación

RELACIÓN “CONOCE” (asociación)

- En caso de duda pueden omitirse las flechas de navegación:

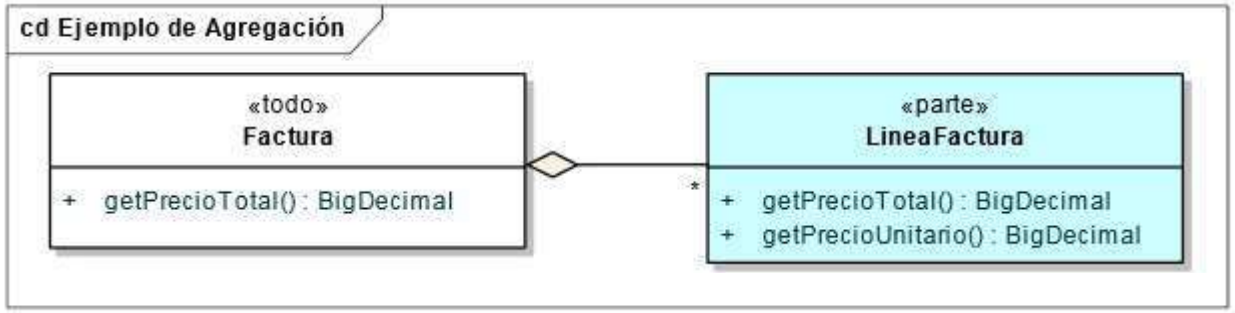


Multiplicidad: Indica cuantos objetos de una clase se relacionan con la otra. En nuestro caso cada Socio tiene varios préstamos y se representa con la *, y a la vez cada préstamo se hace a un único socio, y por eso el 1.

En las asociaciones hay una relación más fuerte entre ambos elementos que trasciende el marco tmporal de una única operación. Aunque cada uno de los objetos sigue teniendo objetivos diferentes.

RELACIÓN “TODO-PARTE” (composición/agregación)

- Es un caso particular de la Asociación, cuando los objetos representan una relación Todo-Parte, por ejemplo: Una factura y sus productos



Tips para documentar un diagrama de clases

- Distinguir las relaciones de asociación y agregación es una sutileza que provee el lenguaje de diseño pero que no aporta demasiado al resultado final de la solución.
- Nosotros trabajaremos con la relación de asociación a secas sin entrar en mayores detalles.
- En un diagrama de clases no puede haber clases que no estén relacionadas con otras